JU-MAP

GRADUATION PROJECT



Table of Contents

01
Introduction

02
Technology Stack

03
Project Timeline and Team Roles

04
Frontend Implementation

05
Backend and Core System Implementation

06
Artificial Intelligence and System Integration

07
Conclusion and Future Work

Introduction

University campuses are large and complex.

Students and visitors can struggle to locate rooms inside buildings.

JU-MAP provides offline indoor navigation for Building B at Jazan University.

Project Timeline - Gantt Chart

The schedule groups development tasks across 14 weeks.

Map, search and routing tasks follow UI/UX design.

Colored bars show task timing and team participation.



Figure: JU-MAP Project Development Timeline (Gantt Chart).

Implementation

Frontend and User Interfaces

Accounts, interactive maps, search, favorites and classroom management.

Backend and Core System Logic

Local Dart services coordinate authentication, search and navigation.

Local Database and Artificial Intelligence Integration

SQLite stores application data. Classical AI-based A* computes routes.

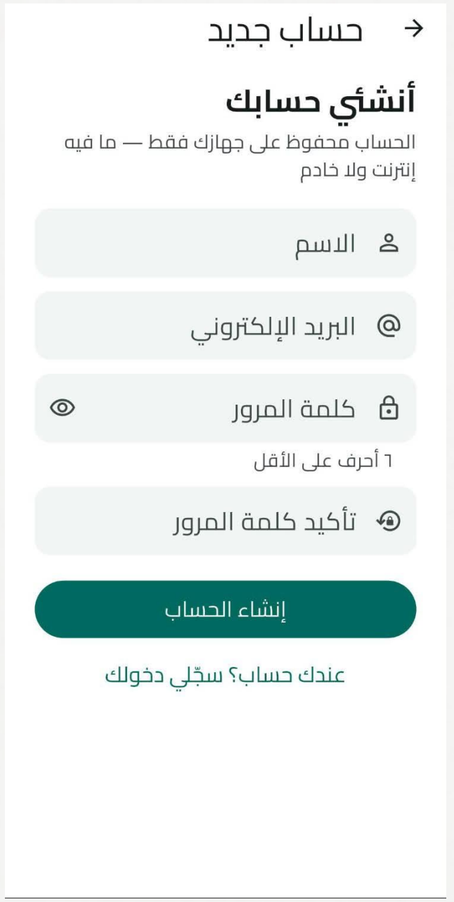
Login and Account Registration

Users create local accounts with name, email and password validation.

Sign-in restores a local session and links each user to personal favorites.



User Login Interface.



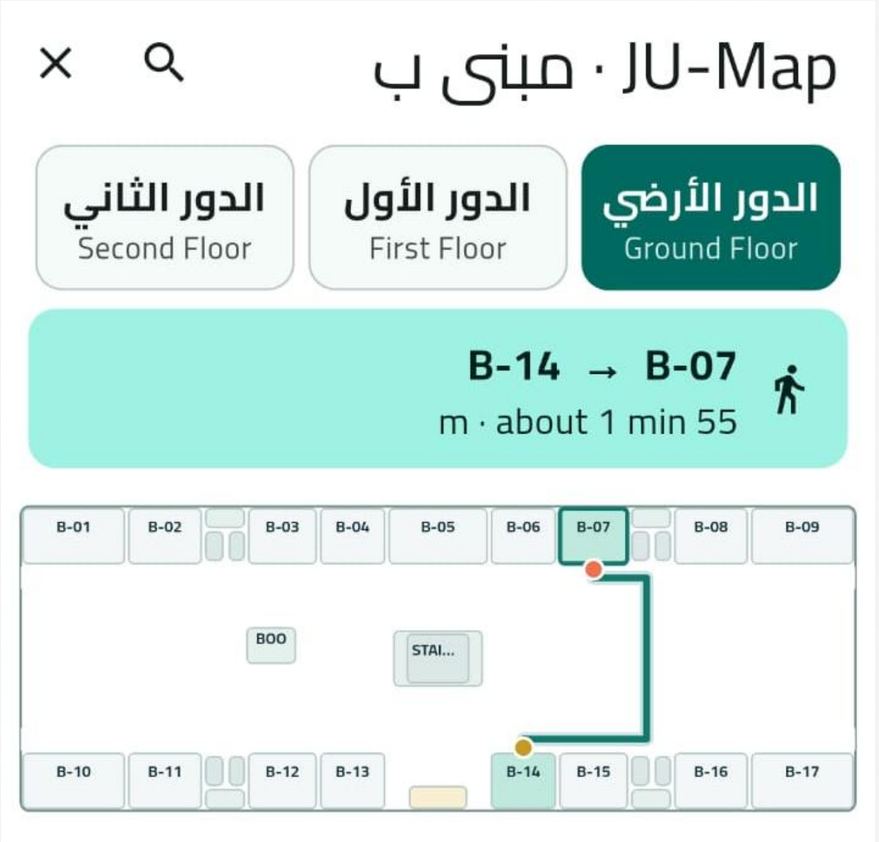
User Registration Interface.

Interactive Map and Floor Navigation

Building B contains ground, first and second floor maps.

Users select rooms and choose a start point and destination.

A highlighted route shows the calculated path with distance and walking-time estimates.



Interactive Building Map and Navigation Route.

Smart Search and Location Details

Arabic normalization and partial names support location search.

Typo tolerance ranks approximate matches.

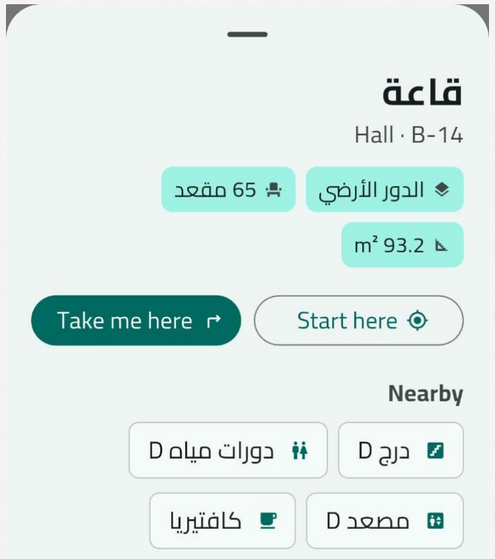
Details show floor, capacity, area and navigation options.



Smart Location Search Interface.



Library Search Result.



Classroom Details and Navigation Options.

Favorite Classrooms

Signed-in users save frequently visited classrooms to their accounts.

The favorites sheet retrieves saved rooms and lets users return to their map locations.

The empty state explains how to add a classroom.



Favorite Classrooms Interface.



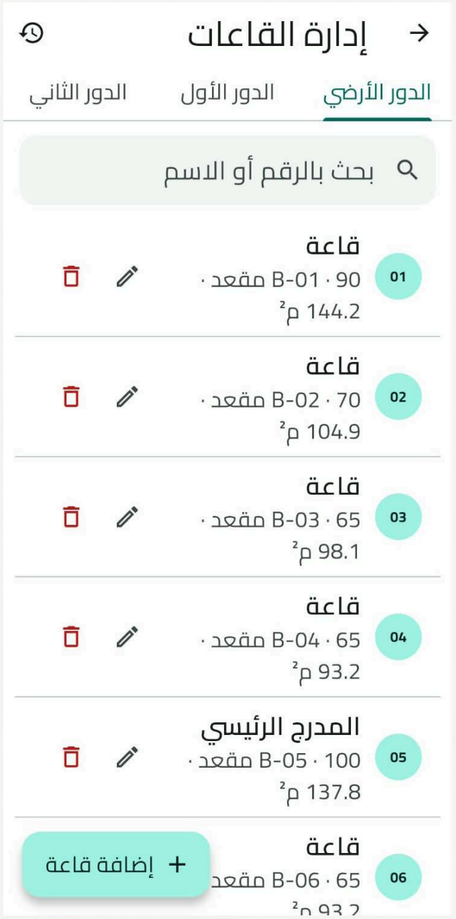
Empty Favorites Interface.

Administrator and Classroom Management

Administrator access supports adding, editing and deleting classroom records.

The editor maintains room codes, bilingual names, type, capacity and area.

Changes also update the navigation graph used for routing.



Administrative Classroom Management.



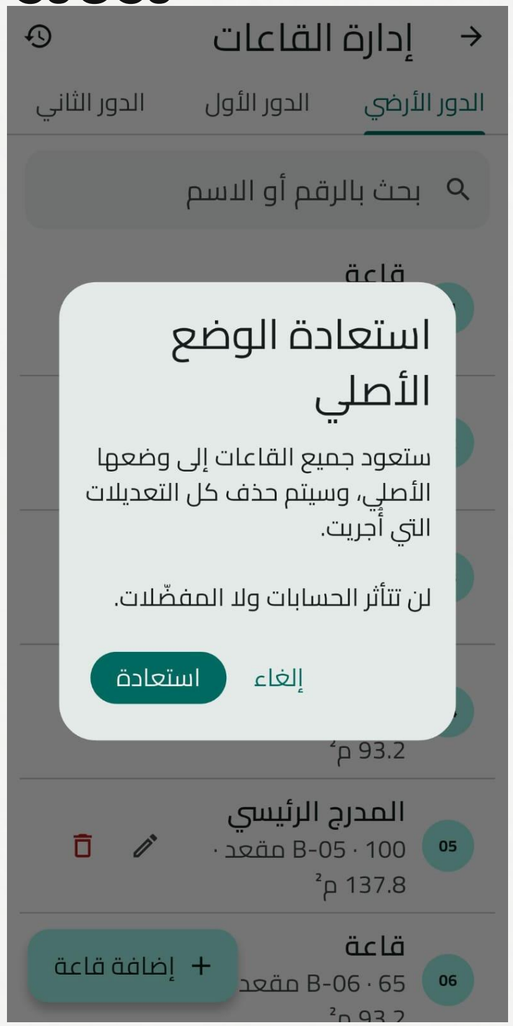
Classroom Information Editing.

Restore Default Classroom Data

A confirmation dialog precedes restoring the original Building B dataset.

The service rebuilds building records and refreshes the cached navigation graph.

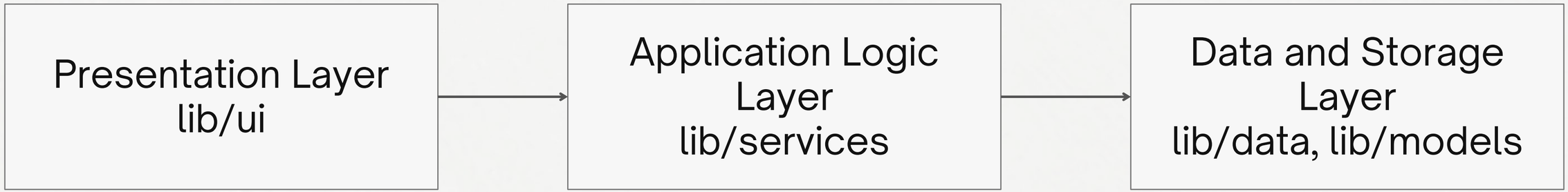
User accounts remain. Favorites linked to replaced or deleted locations may be affected.



Restore Default Classroom Data.

Backend & Core System Implementation

All services and storage run locally on the device.



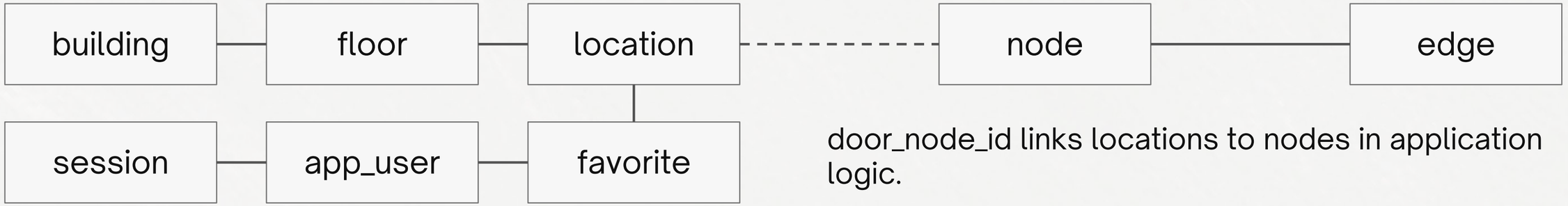
Flutter screens collect input and render results.

Dart services handle accounts, search and routing.

SQLite and models hold building geometry and user data.

Layered Architecture of the Local JU-MAP Application.

SQLite Database and Data Relationships



door_node_id links locations to nodes in application logic.

app_database.dart L87-94

```
await db.execute('''
  CREATE TABLE IF NOT EXISTS favorite(
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER NOT NULL REFERENCES app_user(id) ON DELETE CASCADE,
    location_id INTEGER NOT NULL REFERENCES location(id) ON DELETE CASCADE,
    created_at  INTEGER NOT NULL,
    UNIQUE(user_id, location_id)
  )''');
```

SQLite Database Schema and Data Relationships.

Authentication and Session Management

AuthService validates registration details and verifies passwords at sign-in.

Each account uses a random salt and repeated SHA-256 hashing.

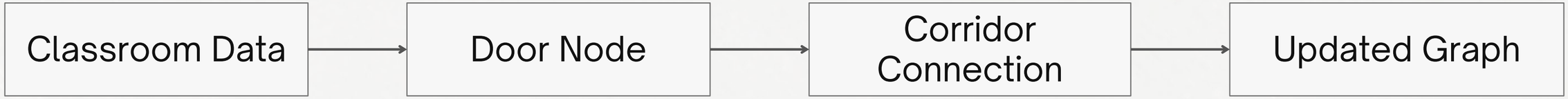
The saved session restores sign-in. User roles control administrator access.

auth_service.dart L61-71

```
final salt = PasswordHasher.newSalt();
final user = await _db.createUser(
  email: e,
  displayName: name,
  salt: salt,
  passwordHash: PasswordHasher.hash(password, salt),
);

_current = user;
await _db.saveSession(user.id);
return AuthResult.success(user);
```

Classroom Management and the Navigation Graph



Door nodes join nearby corridors on the same floor. Edits and deletions update graph data and trigger a cache reload.

admin_service.dart L124-133

```
final nearest = _nearestCorridor(corridor, door.x, door.y);
final dist = math.sqrt(
    math.pow(nearest.x - door.x, 2) + math.pow(nearest.y - door.y, 2));
await _db.insertEdge(GraphEdge(
  id: await _db.nextEdgeId(),
  a: nodeId,
  b: nearest.id,
  weight: dist + 0.5,
  kind: EdgeKind.walk,
));
```

Smart Search Implementation

Arabic normalization removes diacritics and unifies letter variants and numerals.

Partial name and code matches receive relevance scores. Levenshtein Distance handles small spelling differences.

Results are ranked locally using explicit rules.

search_service.dart L167-175

```
if (q.length >= 3) {
  for (final field in [ar, en]) {
    for (final word in field.split(' ')) {
      if (word.length < 3) continue;
      final d = editDistance(q, word);
      if (d <= 2) return 400 - d * 60;
    }
  }
}
```

Exact room codes receive priority over approximate name matches.

Artificial Intelligence: A* Pathfinding Algorithm

$$f(n) = g(n) + h(n)$$

g(n): known travel cost from the start.
h(n): estimated remaining cost.
f(n): estimated total path cost.

A* expands the lowest f(n). Its heuristic combines Euclidean distance with 20 cost units per floor transition.

Classical informed search. No neural network or trained model.

pathfinder.dart L124-124, L131-135

```
static const double _verticalFloor = 20.0;
// ...
double _heuristic(GraphNode a, GraphNode b) {
  final dx = a.x - b.x, dy = a.y - b.y;
  final flat = math.sqrt(dx * dx + dy * dy); // Euclidean, horizontal
  return flat + (a.level - b.level).abs() * _verticalFloor;
}
```

Flutter UI and Core Logic Integration



map_screen.dart loads floors and locations, then initializes routing and search services.

CustomPainter converts spatial data and route nodes into the displayed floor map.

map_screen.dart L67-69, L75-82

```
final floors = await _db.floors();
final all = await _db.allLocations();
final pf = await Pathfinder.load();
// ...
setState(() {
  _floors = floors;
  _all = all;
  _pathfinder = pf;
  _search = SearchService(all);
  _favorites = favs;
  _loading = false;
});
```

Integration Between User Interface, Services, and Local Data.

Conclusion and Future Work

Implemented

Offline indoor navigation for Building B with interactive floor maps and route calculation.

Smart search, local user accounts and favorite classrooms.

Administrative classroom management integrated with classical AI-based A* routing.

Future Work

Expand coverage to additional university buildings.

Strengthen credential protection and administrative access.

Investigate automatic indoor positioning.

Q&A

The End

THANK YOU FOR LISTENING